

## LAB ASSIGNMENT – 9.5

Name: Ch.Vishwanath Rao

Batch:02

Ht no: 2303A51095

### Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):
```

```
    return text[::-1]
```

Task:

1. Write documentation in: (a) Docstring  
(b) Inline comments  
(c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

CODE:

a) DOCSTRING

```
def reverse_string(text):  
    """  
    Reverses the given string.  
  
    Parameters:  
    text (str): The string to be reversed.  
  
    Returns:  
    str: The reversed string.  
    """  
    return text[::-1]  
print(reverse_string("hello")) # Output: "olleh"  
print(reverse_string("Python")) # Output: "nohtyP"  
print(reverse_string("racecar")) # Output: "racecar"  
print(reverse_string.__doc__)
```

## OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai> & C:/Python314/python.exe "c:/Users/chvis/OneDrive/Desktop/ai/all git pdfs/LAB 9.f
• olleh
nohtyp
racecar

Reverses the given string.

Parameters:
text (str): The string to be reversed.

Returns:
str: The reversed string.
```

## b) Inline Comments

```
def reverse_string(text):
    # convert the string to a list of characters
    char_list = list(text)
    # reverse the list of characters
    char_list.reverse()
    # convert the list of characters back to a string
    reversed_text = ''.join(char_list)
    return text[::-1]
# test the function
print(reverse_string("hello world"))
print(reverse_string("racecar"))
```

## c) Google Style Documentation

```
def reverse_string(text:int) -> int:
    """Reverse the given string.

    Args:
        text: The string to reverse.

    Returns:
        The reversed string.

    Raises:
        TypeError: If the input is not a string.
    """
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text[::-1]
```

## Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):  
  
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

CODE:

a) Docstring and Google\_style documentation:

```
9  
10 def password_strength_checker(password:str) -> str:  
11     """  
12     Docstring for password_strength_checker  
13     :param: password (str)  
14     :return: str indicating the strength of the password ("Weak", "Moderate", "Strong")  
15     :exceptions: ValueError for invalid input types or values  
16     :error handling: Catches ValueError and prompts user to enter valid input  
17     :side effects: None  
18     :description: Evaluates the strength of a given password based on its length and character composition.  
19     | Validates the input to ensure that it is a string and meets certain criteria for strength.  
20     :example usage:  
21     | password_strength_checker("P@ssw0rd")  
22     | Returns: "Strong"  
23     """  
24     if not isinstance(password, str):  
25         raise ValueError("Password must be a string.")  
26  
27     length = len(password)  
28     has_upper = any(c.isupper() for c in password)  
29     has_lower = any(c.islower() for c in password)  
30     has_digit = any(c.isdigit() for c in password)  
31     has_special = any(not c.isalnum() for c in password)  
32  
33     if length < 6:  
34         return "Weak"  
35     elif length < 12:  
36         if has_upper and has_lower and has_digit:  
37             return "Moderate"  
38         else:  
39             return "Weak"  
40     else:  
41         if has_upper and has_lower and has_digit and has_special:  
42             return "Strong"  
43         else:  
44             return "Moderate"  
45     password = "P@ssw0rd"  
46     print(password_strength_checker(password))  
47
```

## b) Inline comments

```
def check_strength(password):
    # Check if password is at least 8 characters long
    if len(password) < 8:
        # Return False if password is too short\
        return False
    # Check if password contains at least one uppercase letter, one lowercase letter, and one number
    if not any(char.isupper() for char in password):
        return False
    # Check if password contains at least one special character
    if not any(char in "!@#$%^&*()_+-=[]{}|;:,.<>?/" for char in password):
        return False
    # If all checks pass, return True
    return True

# Test the function
print(check_strength("Abc123!")) # True
```

## Problem 3: Math Utilities Module

Task:

1. Create a module math\_utils.py with functions:
  - o square(n)
  - o cube(n)
  - o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

## CODE:

math\_utils.py > ...

```
1 def factorial(n):
2     """
3     Calculate the factorial of a non-negative integer n.
4     :param n: A non-negative integer
5     :return: The factorial of n
6     :raises ValueError: If n is negative or not an integer
7
8
9     Example usage:
10    print(factorial(5)) # Output: 120
11
12    """
13    if not isinstance(n, int):
14        raise ValueError("Input must be an integer.")
15    if n < 0:
16        raise ValueError("Input must be a non-negative integer.")
17
18    result = 1
19    for i in range(2, n + 1):
20        result *= i
21
22    return result
23 def square(x):
24     """
25     Calculate the square of a number x.
26     :param x: A number (int or float)
27     :return: The square of x
28     :raises ValueError: If x is not a number
29
30
31     Example usage:
32    print(square(4)) # Output: 16
33    print(square(2.5)) # Output: 6.25
34
35    """
36    if not isinstance(x, (int, float)):
37        raise ValueError("Input must be a number.")
38
39    return x * x
40 def cube(x):
41     """
42     Calculate the cube of a number x.
43     :param x: A number (int or float)
44     :return: The cube of x
45     :raises ValueError: If x is not a number
46
47
48     Example usage:
49    print(cube(3)) # Output: 27
50    print(cube(1.5)) # Output: 3.375
51
52    """
53    if not isinstance(x, (int, float)):
54        raise ValueError("Input must be a number.")
55
56    return x * x * x
57 print(factorial(5))
58 print(square(4))
59 print(cube(3))
60
```

OUTPUT:

### Functions

#### `cube(x)`

Calculate the cube of a number x.  
:param x: A number (int or float)  
:return: The cube of x  
:raises ValueError: If x is not a number

Example usage:

```
print(cube(3)) # Output: 27  
print(cube(1.5)) # Output: 3.375
```

#### `factorial(n)`

Calculate the factorial of a non-negative integer n.  
:param n: A non-negative integer  
:return: The factorial of n  
:raises ValueError: If n is negative or not an integer

Example usage:

```
print(factorial(5)) # Output: 120
```

#### `square(x)`

Calculate the square of a number x.  
:param x: A number (int or float)  
:return: The square of x  
:raises ValueError: If x is not a number

Example usage:

```
print(square(4)) # Output: 16  
print(square(2.5)) # Output: 6.25
```

120

16

27

wrote math\_utils.html

## Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:
  - o mark\_present(student)
  - o mark\_absent(student)
  - o get\_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

CODE:

```
attendance.py > ...
1  class Attendance:
2      def __init__(self, student_name, date, status):
3          """Docstring for Attendance class
4          :param: student_name (str), date (str), status (str)
5          :return: None
6          :exceptions: ValueError for invalid input types or values
7          :error handling: Catches ValueError and prompts user to enter valid input
8          :side effects: None
9          :description: Initializes an Attendance object with the given student name, date, and status.
10             Validates the input to ensure that student_name and date are strings and status is either "Present" or "Absent".
11          :example usage:
12             attendance = Attendance("John Doe", "2024-06-01", "Present")
13          """
14          if not isinstance(student_name, str) or not isinstance(date, str):
15              raise ValueError("Student name and date must be strings.")
16          if status not in ["Present", "Absent"]:
17              raise ValueError("Status must be either 'Present' or 'Absent'.")
18
19          self.student_name = student_name
20          self.date = date
21          self.status = status
22      def __str__(self):
23          """Docstring for __str__ method
24          :param: None
25          :return: str representation of the Attendance object
26          :exceptions: None
27          :error handling: None
28          :side effects: None
29          :description: Returns a string representation of the Attendance object in the format "Student Name:
30             Date: YYYY-MM-DD, Status: Present/Absent".
31          :example usage:
32             attendance = Attendance("John Doe", "2024-06-01", "Present")
33             print(attendance) # Output: Student Name: John Doe, Date: 2024-06-01, Status: Present
34          """
35          return f"Student Name: {self.student_name}, Date: {self.date}, Status: {self.status}"
36  attendance = Attendance("John Doe", "2024-06-01", "Present")
37  print(attendance)
38
```

OUTPUT:

```
attendance
CLASSES
builtins.object
Attendance

class Attendance(builtins.object)
|   Attendance(student_name, date, status)
|
|   Methods defined here:
|
|   __init__(self, student_name, date, status)
|       Docstring for Attendance class
|       :param: student_name (str), date (str), status (str)
|       :return: None
|       :exceptions: ValueError for invalid input types or values
|       :error handling: Catches ValueError and prompts user to enter valid input
|       :side effects: None
|       :description: Initializes an Attendance object with the given student name, date, and status.
|                   Validates the input to ensure that student_name and date are strings and status is either "Present" or "Absent".
|       :example usage:
|           attendance = Attendance("John Doe", "2024-06-01", "Present")
|
|   __str__(self)
|       Docstring for __str__ method
|       :params: None
|       :return: str representation of the Attendance object
|       :exceptions: None
|       :error handling: None
|       :side effects: None
|       :description: Returns a string representation of the Attendance object in the format "Student Name:
|                     Date: YYYY-MM-DD, Status: Present/Absent".
|       :example usage:
|           attendance = Attendance("John Doe", "2024-06-01", "Present")
|           print(attendance) # Output: Student Name: John Doe, Date: 2024-06-01, Status: Present
|
-----
Data descriptors defined here:
..dict_
dictionary for instance variables
..weakref_
list of weak references to the object
:side effects: None
:description: Returns a string representation of the Attendance object in the format "Student Name:
Date: YYYY-MM-DD, Status: Present/Absent".
:example usage:
attendance = Attendance("John Doe", "2024-06-01", "Present")
print(attendance) # Output: Student Name: John Doe, Date: 2024-06-01, Status: Present
-----
Data descriptors defined here:
..dict_
dictionary for instance variables
..weakref_
list of weak references to the object
>Date: YYYY-MM-DD, Status: Present/Absent".
:example usage:
attendance = Attendance("John Doe", "2024-06-01", "Present")
print(attendance) # Output: Student Name: John Doe, Date: 2024-06-01, Status: Present
-----
Data descriptors defined here:
..dict_
dictionary for instance variables
..weakref_
list of weak references to the object
print(attendance) # Output: Student Name: John Doe, Date: 2024-06-01, Status: Present
-----
Data descriptors defined here:
..dict_
dictionary for instance variables
..weakref_
list of weak references to the object
-----
Data descriptors defined here:
..dict_
dictionary for instance variables
..weakref_
list of weak references to the object
-----
Data descriptors defined here:
..dict_
dictionary for instance variables
..weakref_
list of weak references to the object
DATA
```

```
Data
attendance = <attendance.Attendance object>
```

```
Data
attendance = <attendance.Attendance object>
attendance = <attendance.Attendance object>
```



## Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

CODE:

```
27
28 def read_file(filename:str) -> str:
29     """
30     Reads the contents of a file and returns it as a string.
31     :param filename: The name of the file to read
32     :return: The contents of the file as a string
33     :raises ValueError: If the filename is not a string
34     :raises FileNotFoundError: If the file does not exist
35     :raises IOError: If there is an error reading the file
36     Example usage:
37     print(read_file("example.txt"))
38
39     """
40     if not isinstance(filename, str):
41         raise ValueError("Filename must be a string.")
42
43     try:
44         with open(filename, 'r') as file:
45             contents = file.read()
46             return contents
47     except FileNotFoundError:
48         raise FileNotFoundError(f"The file '{filename}' does not exist.")
49     except IOError as e:
50         raise IOError(f"An error occurred while reading the file: {e}")
51
52     try:
53         print(read_file("example.txt"))
54     except ValueError as ve:
55         print(f"ValueError: {ve}")
56     except FileNotFoundError as fnfe:
57         print(f"FileNotFoundError: {fnfe}")
58     except IOError as ioe:
59         print(f"IOError: {ioe}")
60
61     print(read_file("example.txt"))
62
```

OUTPUT:

```
hello world!.
Help on module assg_09_5:

NAME
    assg_09_5

FUNCTIONS
    read_file(filename: str) -> str
        Reads the contents of a file and returns it as a string.
        :param filename: The name of the file to read
        :return: The contents of the file as a string
        :raises ValueError: If the filename is not a string
        :raises FileNotFoundError: If the file does not exist
        :raises IOError: If there is an error reading the file

FILE
```

## Functions

```
read_file(filename: str) -> str
    Reads the contents of a file and returns it as a string.
    :param filename: The name of the file to read
    :return: The contents of the file as a string
    :raises ValueError: If the filename is not a string
    :raises FileNotFoundError: If the file does not exist
    :raises IOError: If there is an error reading the file
```

Python 3.14.2 [tags/v3.14.2:df79316, MSC v.1944 64 bit (AMD64)]  
Windows-11

Module Ins

assg\_09\_5

c:\users\

## Functions

```
read_file(filename: str) -> str
    Reads the contents of a file and returns it as a string.
    :param filename: The name of the file to read
    :return: The contents of the file as a string
    :raises ValueError: If the filename is not a string
    :raises FileNotFoundError: If the file does not exist
    :raises IOError: If there is an error reading the file
```

## INLINE COMMENTS:

```
assg_09_5.py > ...
1  # Define a function that reads a file and returns its contents as a string
2  def read_file(filename: str) -> str:
3      """
4      Reads the contents of a file and returns it as a string.
5      :param filename: The name of the file to read
6      :return: The contents of the file as a string
7      :raises ValueError: If the filename is not a string
8      :raises FileNotFoundError: If the file does not exist
9      :raises IOError: If there is an error reading the file
10     """
11     # Check if the filename parameter is a string, raise ValueError if not
12     if not isinstance(filename, str):
13         raise ValueError("Filename must be a string.")
14
15     # Try to open and read the file
16     try:
17         # Open the file in read mode
18         with open(filename, 'r') as file:
19             # Read the entire file contents into a string variable
20             contents = file.read()
21             # Return the file contents
22             return contents
23     # Catch FileNotFoundError if the file doesn't exist
24     except FileNotFoundError:
25         raise FileNotFoundError(f"The file '{filename}' does not exist.")
26     # Catch IOError for any other file reading errors
27     except IOError as e:
28         raise IOError(f"An error occurred while reading the file: {e}")
29
30
31 # Attempt to read example.txt and print its contents
32 try:
33     print(read_file("example.txt"))
34 # Handle ValueError if filename is not a string
35 except ValueError as ve:
36     print(f"ValueError: {ve}")
37 # Handle FileNotFoundError if the file doesn't exist
38 except FileNotFoundError as fnfe:
39     print(f"FileNotFoundError: {fnfe}")
40 # Handle IOError for any file reading errors
41 except IOError as ioe:
42     print(f"IOError: {ioe}")
43
```